Dione: A Sovereign Large Action Model Orchestration Engine with Intent-Aware Poset Lattice Safety, $\frac{1}{2}$-Approximation Knapsack Context Allocation, and Closed-Loop Visual Grounding

Jass Shivnani **Riya Khialani** **Alex**

Under the Guidance of **Prof. Geocey Shejy**

Department of Computer Engineering, Vivekanand Education Society's Institute of Technology (VESIT)

jass.shivnani@ves.ac.in

November 2026

Abstract

Autonomous Large Action Models (LAMs) that operate directly on a user's personal operating system and external cloud services face four fundamental systems challenges: (1) bounded context capacity when assembling heterogeneous personal knowledge graphs and episodic memories, (2) vulnerability to indirect prompt injection and destructive side effects when executing privileged OS or cross-service actions, (3) non-terminating action cycles in multi-step tool execution, and (4) open-loop hallucination during graphical user interface (GUI) automation. In this paper, we present **Dione**, a sovereign, local-first LAM orchestration engine implemented across 142 source modules (30,270 lines of code) that unifies native Windows 11 desktop automation (`cua-driver` Set-of-Mark and Win32 GDI/UIA grounding), a 2-hop NetworkX relational knowledge multigraph, and 8 external service integrations (Gmail, Google Calendar, Google Drive, Google Photos, WhatsApp, Telegram, Slack, and Instagram) under a single Google OAuth2 Single-Sign-On (SSO) vault. We formalize four core algorithmic contributions: (i) a greedy density-ordered **0-1 Knapsack Context Allocator** with a single-item safeguard that guarantees $R(S^*) \geq \frac{1}{2}R(\mathrm{OPT})$ in $\mathcal{O}(M \log M)$ time; (ii) a **5-Tier Partially Ordered Set (Poset) Permission Lattice** with intent-verified autonomy that distinguishes user-commanded external communications from unprompted indirect-injection exfiltration attempts; (iii) a temporal-decay **Maximal Marginal Relevance (MMR)** hybrid memory retriever alongside a SHA-256 deterministic loop circuit breaker; and (iv) a **Closed-Loop Visual Goal Verifier** paired with a multi-model rate-limit failover chain across Google AI Studio models. Empirical evaluation on the production Dione runtime demonstrates that our Knapsack allocator achieves **99.26%–100.00%** of the exact Dynamic Programming (DP) optimal reward while running **30.3×–49.0× faster** (74.02 μs at $M = 100$) and yielding up to **+556.7%** higher relevance reward than FIFO sliding windows. The 5-Tier Safety Kernel achieves **100.0%** gate classification accuracy across 500 evaluations with a mean overhead of **36.18 μs** per tool call, while MMR retrieval improves intra-list semantic diversity by **+320.5%–344.5%** over standard top-k cosine retrieval.

Keywords: Large Action Models, Computer-Use Agents, ReAct Orchestration, 0-1 Knapsack Context Allocation, Poset Permission Lattice, Indirect Prompt Injection Defense, GraphRAG, Maximal Marginal Relevance.

1 Introduction: Bridging Open-Loop Chatbots and Sovereign OS Execution

Recent advances in foundation models have shifted artificial intelligence from passive text generation toward autonomous *Large Action Models* (LAMs) capable of invoking system tools, navigating graphical user interfaces (GUIs), and coordinating multi-service workflows [1, 2, 3]. Despite promising capabilities demonstrated in isolated benchmarks such as OSWorld [2] and Set-of-Mark (SoM) visual grounding [4], deploying an always-on executive assistant on a user's primary workstation exposes critical failure modes:

1. **Context Starvation vs. Redundancy in Personal Memory:** Naive first-in-first-out (FIFO) sliding windows or unconstrained top-k vector lookups flood the prompt with near-duplicate conversational turns while dropping high-density relational facts (e.g., collaborator email addresses, project deadlines, and social commitments).
2. **The Autonomy–Safety Dilemma:** Existing desktop agents either interrupt the user with confirmation dialogs on every trivial read/click operation—destroying autonomous utility—or execute with unrestricted shell privileges, leaving the workstation vulnerable to catastrophic commands (`rm -rf`, registry deletion) and indirect prompt injection exfiltration via email or messaging APIs.
3. **Open-Loop GUI Hallucination & Action Stagnation:** When an agent launches a desktop application, clicks a coordinate, or dispatches a browser action, standard ReAct loops [1] assume success from the tool's return code without verifying whether the foreground window state actually transitioned on screen, often falling into repetitive single-tool or alternating $A \leftrightarrow B$ ping-pong loops.

4. **Fragmented Cloud Service Authentication & Quota Fragility:** Real-world executive tasks span local files, desktop windows, and cloud services (Gmail, Calendar, Drive, Photos, WhatsApp, Telegram, Slack, Instagram). Disconnected credential stores and per-project API rate limits (Requests Per Minute [RPM] and Requests Per Day [RPD]) frequently stall multi-step execution mid-workflow.

To resolve these challenges, we design and implement **Dione**, a sovereign LAM orchestration engine. Every subsystem in Dione adheres to a strict *Zero-Mock Policy*: all OS operations execute genuine Win32/GDI/UIA system calls via `cua-driver`, all graph traversals run on a persisted NetworkX multigraph, and all external services execute live REST/IMAP/SMTP/Baileys/Telegram protocol calls.

Core Contributions: (1) A full-stack sovereign LAM architecture spanning 142 modules (30,270 non-empty LOC) with unified Google OAuth2 SSO and 24 typed service tools across 8 platforms; (2) A provably $\frac{1}{2}$-optimal $\mathcal{O}(M \log M)$ Knapsack Context Allocator that empirically captures $\geq 99.26\%$ of exact DP reward at $< 350\,\mu$s latency; (3) An Intent-Aware 5-Tier Poset Permission Lattice that auto-approves user-commanded actions while intercepting 100% of unprompted exfiltration and destructive operations in $36.18\,\mu$s; and (4) A reproducible, zero-mock empirical benchmark suite (`dione_research_benchmark.py`) bundled directly inside the repository.

2 System Architecture & Codebase Telemetry

Dione is architected as a modular, event-driven runtime coupling a FastAPI + AG-UI (Agent-User Interaction Protocol) asynchronous server, a React TypeScript studio interface, a full-duplex Gemini Live 24 kHz neural voice frontman, and a native Windows 11 computer-use substrate. Table 1 reports the exact source file and non-empty lines-of-code (LOC) distribution measured directly from the production repository by `dione_research_benchmark.py`.

Table 1: Empirical Codebase Architecture Telemetry Across Dione Subsystems (Measured via `dione_research_benchmark.py`)

Dione Architectural Subsystem	**Source Files**	**Non-Empty LOC**	**Primary Responsibilities & Protocols**
Core Orchestration & Safety (`server/core`)	20	4,431	ReAct loop, Knapsack RAG, 5-Tier Lattice, Loop Breaker
Plugins & 8 Service Integrations (`server/plugins`)	27	5,986	Unified Google OAuth2, Gmail, Calendar, Drive, Photos, Slack
React TypeScript Studio UI (`client/src`)	24	5,317	AG-UI SSE stream, DAG viewer, Vision HUD, Integrations Hub
Messaging Bridges & CLI (`whatsapp`, `telegram`, `cli`)	16	3,849	Node.js Baileys WhatsApp Web bridge, Telegram long-polling
Computer-Use & CUA Driver (`hermes_cua`)	14	3,820	`cua-driver` 0.21.0, Set-of-Mark (SoM), Win32 GDI & UIA
FastAPI & AG-UI Gateway (`server/api`)	13	1,936	REST, WebSocket, AG-UI SSE, OpenAI-compatible `/v1` gateway
LLM Adapters & Live Audio (`server/llm`)	12	1,573	Multi-model failover chain, Gemini Live 24 kHz PCM duplex
Knowledge Multigraph & Social (`server/knowledge`)	7	1,571	NetworkX MultiDiGraph, 2-hop GraphRAG, Commitment Mining
Proactive Dream & Heartbeat (`server/proactive`)	4	1,072	Win32 `GetLastInputInfo` idle detector, background synthesis
Memory, Vector & MMR Search (`server/memory`)	5	715	ChromaDB vector store, exponential temporal decay, MMR
Total Production Codebase	**142**	**30,270**	**8 Connected Services / 24 External + 18 Built-in Tools**

2.1 Unified Google OAuth2 Vault & Multi-Service Tool Synthesis

To eliminate credential fragmentation across cloud tools, Dione introduces a unified OAuth2 manager (`google_oauth.py`) backed by `CredentialVault`. A single authorization flow requests the composite scope set:

$$\mathscr{S}_{\text{Google}} = \{\texttt{gmail.modify}, \texttt{gmail.send}, \texttt{gmail.readonly}, \texttt{calendar}, \texttt{calendar.events}, \texttt{drive}, \texttt{drive.file}, \texttt{photoslibrary.readonly}\}.$$

Upon token acquisition or automatic `refresh_token` rotation, `IntegrationRegistry` propagates the credential across all four Google Workspace adapters (`google_mail`, `google_calendar`, `google_drive`, and `google_photos`). Alongside token/device-code adapters for `whatsapp`, `telegram`, `slack`, and `instagram`, the registry dynamically synthesizes 24 typed Python `inspect.Signature` callables via `get_dione_tools_map()` so the language model invokes external cloud APIs and local OS tools within a single unified tool namespace.

3 Mathematical Formulations & Core Algorithms

3.1 $\frac{1}{2}$-Approximation 0-1 Knapsack Context Allocation

Let $\mathscr{I} = \{1, 2, \ldots, M\}$ denote the set of candidate context chunks retrieved from the 2-hop Knowledge Multigraph and long-term vector memory for user query q. Each candidate item $i \in \mathscr{I}$ possesses a relevance reward $r_i \in (0, 1]$ and a token footprint cost $c_i \in \mathbb{Z}^+$ (estimated as $\max(1, \lfloor \text{len}(\text{text}_i)/4 \rfloor)$). Given a strict system-instruction context budget $C_{\text{budget}} \in \mathbb{Z}^+$ (default $C_{\text{budget}} = 6656$ tokens), the optimal context packing problem is the 0-1 Knapsack problem:

$$\max_{x \in \{0,1\}^M} \sum_{i=1}^{M} r_i x_i \quad \text{subject to} \quad \sum_{i=1}^{M} c_i x_i \leq C_{\text{budget}}. \tag{1}$$

Because exact Dynamic Programming requires $\mathscr{O}(M \cdot C_{\text{budget}})$ pseudo-polynomial time, invoking exact DP on every turn adds multi-millisecond latency. Instead, Dione's `allocate_context_knapsack` filters feasible items $\mathscr{I}_{\text{feas}} = \{i \in \mathscr{I} : 0 < c_i \leq C_{\text{budget}}\}$, computes the reward-to-token density $\rho_i = r_i / c_i$, sorts $\mathscr{I}_{\text{feas}}$ in non-increasing order $\rho_{\pi(1)} \geq \rho_{\pi(2)} \geq \cdots \geq \rho_{\pi(M')}$, and greedily packs items into S_{greedy} while maintaining running cost $\leq C_{\text{budget}}$. To prevent arbitrarily poor performance when a single large, high-reward chunk is blocked by tiny high-density fragments, Dione evaluates the single-item safeguard $i^* = \arg\max_{i \in \mathscr{I}_{\text{feas}}} r_i$ and selects:

$$S^* = \arg \max_{S \in \{S_{\text{greedy}}, \{i^*\}\}} R(S), \quad \text{where } R(S) = \sum_{i \in S} r_i. \tag{2}$$

Theorem 1 ($\frac{1}{2}$-Approximation Guarantee [6])**.** *For any feasible instance of the context allocation problem* (1)*, the set S^* returned by Equation* (2) *runs in $\mathscr{O}(M \log M)$ time and satisfies $R(S^*) \geq \frac{1}{2} R(OPT)$.*

Proof. Let k be the first index in the density-sorted order π that cannot fit into S_{greedy} because $\sum_{j=1}^{k} c_{\pi(j)} > C_{\text{budget}}$. If no such k exists, $S_{\text{greedy}} = \mathscr{I}_{\text{feas}}$ and $R(S^*) = R(\text{OPT})$. Otherwise, the fractional knapsack optimum $R(\text{LP})$ upper-bounds $R(\text{OPT})$ and satisfies $R(\text{OPT}) \leq R(\text{LP}) < \sum_{j=1}^{k-1} r_{\pi(j)} + r_{\pi(k)} \leq R(S_{\text{greedy}}) + r_{i^*}$. Therefore, $\max\{R(S_{\text{greedy}}), r_{i^*}\} \geq \frac{1}{2}\big(R(S_{\text{greedy}}) + r_{i^*}\big) > \frac{1}{2} R(\text{OPT})$. □

3.2 Temporal-Decay Maximal Marginal Relevance (MMR) Memory Retrieval

To prevent conversational memory retrieval from returning k redundant paraphrases of the same recent turn while preserving evergreen user identity facts, Dione combines exponential temporal decay with Maximal Marginal Relevance [5] in `server/memory/hybrid_search.py`. Given a memory item i with age Δt_i (in days) and half-life $T_{1/2} = 30$ days, the temporal decay multiplier γ_i is defined as:

$$\gamma_i(\Delta t_i) = \begin{cases} 1.0, & \text{if } \texttt{is_evergreen}(i) = \text{True}, \\ \exp\left(-\frac{\ln 2}{T_{1/2}} \max(0, \Delta t_i)\right), & \text{otherwise.} \end{cases} \tag{3}$$

At each selection step $t \in \{1, \ldots, k\}$, let $\mathscr{S}_{t-1}$ denote the already-selected memory set and $\mathscr{U}_{t-1} = \mathscr{C} \setminus \mathscr{S}_{t-1}$ the remaining candidates. Dione selects the next memory i_t^* maximizing the decayed MMR objective with trade-off parameter $\lambda = 0.65$:

$$i_t^* = \arg \max_{i \in \mathscr{U}_{t-1}} \left[\lambda \cdot \big(\cos(\mathbf{e}_q, \mathbf{e}_i) \cdot \gamma_i(\Delta t_i)\big) - (1 - \lambda) \max_{j \in \mathscr{S}_{t-1}} \cos(\mathbf{e}_i, \mathbf{e}_j) \right]. \tag{4}$$

3.3 5-Tier Poset Permission Lattice & Intent-Verified Autonomy

Following Denning's lattice model of secure information flow [7], Dione's `SafetyKernel` (`server/core/safety.py`) defines a totally ordered 5-Tier Permission Poset $(\mathscr{L}, \leq, \vee, \wedge)$:

$$\text{NONE } (0) < \text{READ } (1) < \text{INSPECT } (2) < \text{EXECUTE } (3) < \text{PRIVILEGED } (4) < \text{CATASTROPHIC } (5), \tag{5}$$

with join $a \vee b = \max(a,b)$ and meet $a \wedge b = \min(a,b)$. Let $\tau_{\text{auto}} = \text{EXECUTE } (3)$ denote the maximum autonomous execution tier. A major vulnerability in multi-service agents is *indirect prompt injection*: while reading an untrusted webpage or document, adversarial text may instruct the agent to call `gmail_send` or `whatsapp_send` to exfiltrate local data. Dione mitigates this via an **Intent-Aware Classification Function** $\Phi(t, \theta, u) \to \mathscr{L}$ that inspects both the tool call (t, θ) and the user's originating prompt u:

$$\Phi(t,\theta,u) = \begin{cases} \text{CATASTROPHIC } (5), & \text{if } (t,\theta) \text{ matches } \mathscr{R}_{\text{cat}} \cup \mathscr{R}_{\text{path}} \quad \text{(Hard Block)}, \\ \text{PRIVILEGED } (4), & \text{if } \theta \text{ matches } \mathscr{R}_{\text{inj}} \text{ or } t \in \mathscr{T}_{\text{delete/kill}} \quad \text{(1-Click Confirmation Gate)}, \\ \text{PRIVILEGED } (4), & \text{if } t \in \mathscr{T}_{\text{comm}} \text{ and } \mathscr{K}_{\text{comm}} \cap \text{tokens}(u) = \emptyset \quad \text{(Unprompted Send Gate)}, \\ \text{EXECUTE } (3), & \text{if } t \in \mathscr{T}_{\text{comm}} \text{ and } \mathscr{K}_{\text{comm}} \cap \text{tokens}(u) \neq \emptyset \quad \text{(Intent-Verified Auto)}, \\ \text{EXECUTE } (3), & \text{if } t \in \mathscr{T}_{\text{desktop/calendar/drive}} \quad \text{(Autonomous Execution)}, \\ \text{INSPECT } (2) \text{ or READ } (1), & \text{otherwise.} \end{cases} \tag{6}$$

Here, $\mathscr{T}_{\text{comm}} = \{\texttt{gmail_send}, \texttt{whatsapp_send}, \texttt{telegram_send}, \texttt{slack_send}, \texttt{send_email}\}$ and $\mathscr{K}_{\text{comm}}$ is the set of explicit communication intent lexemes (`send`, `email`, `message`, `reply`, `share`, `whatsapp`, `telegram`, `slack`, `notify`). Thus, when a user explicitly commands Dione to message or email a contact, execution proceeds with zero friction ($\Phi = 3 \leq \tau_{\text{auto}}$); if an external send is triggered without communication intent in u, Φ escalates to Tier 4, pausing the async event loop ('asyncio.Event') and surfacing an interactive approval card.

3.4 Closed-Loop Visual Goal Verification & Loop Circuit Breaking

To eliminate open-loop GUI failures, Dione wraps every desktop and browser tool invocation in a two-stage verification harness:

1. **Deterministic Loop Circuit Breaker (`ToolLoopDetector`):** Every tool invocation (t_n, θ_n) at step n is canonicalized via sorted-key JSON serialization and hashed to a 64-bit SHA-256 prefix $h_n = \text{SHA256}(t_n \,\|\, \text{sort_json}(\theta_n))_{1:16}$. The detector halts execution if: (i) *Generic Repeat*: $h_n = h_{n-1} = h_{n-2}$ ($k_{\text{rep}} = 3$); (ii) *Alternating Ping-Pong*: $h_{n-2j} = h_n$ and $h_{n-2j-1} = h_{n-1}$ with $h_n \neq h_{n-1}$ across $2p = 8$ steps; or (iii) *Global Turn Ceiling*: $n \geq C_{\max} = 25$.
2. **Post-Action Visual Audit (`GoalVerifier`):** Following any desktop action, Dione pauses 500 ms for OS window rendering, captures the desktop frame and foreground Win32 window handle (`HWND`, title, bounding rectangle, fullscreen state), and evaluates empirical goal predicates $V(u, t_n, \theta_n, \text{audit}_n) \in \{\text{True}, \text{False}\}$. If $V = \text{False}$ (e.g., a browser landed on a YouTube search results page `/results?search_query=...` instead of a direct video watch page, or failed to enter fullscreen), `DioneEngine` injects a structured `[Closed-Loop Visual Reflection Directive]` into the chat session (up to $R_{\max} = 2$ reflection cycles) forcing the model to diagnose the discrepancy and issue a corrective tool call before completing the turn.

4 Empirical Evaluation (Zero-Mock Benchmarks)

All experiments were executed directly against the production Dione codebase using `dione_research_benchmark.py` on Python 3.11.2 (`x86_64`). No simulated delays or synthetic metrics were used.

4.1 Experiment 1: 0-1 Knapsack Context Allocator vs. Exact DP and FIFO

We evaluated `allocate_context_knapsack` across candidate pool sizes $M \in \{10, 25, 50, 100, 250, 500\}$ under the production token budget $C_{\text{budget}} = 6656$, averaging over 50 random trials per pool size (300 total instances). We compared Dione's $\mathscr{O}(M \log M)$ allocator against: (1) Exact Dynamic Programming (OPT, discretized at 16-token steps) and (2) Naive FIFO chronological packing.

As shown in Table 2, while Theorem 1 guarantees a worst-case bound of 50%, Dione's density-greedy + single-item safeguard achieves **99.26% to 100.00%** of the exact DP optimal reward across all tested pool sizes. At $M = 100$ candidates, Dione completes allocation in **74.02 μs** (49.0× faster than Exact DP's 3,628.95 μs) while increasing total relevance reward packed into the prompt by **+228.3%** over FIFO truncation (and **+556.7%** at $M = 500$).

Table 2: Empirical Performance of Dione's 0-1 Knapsack Context Allocator ($C_{\text{budget}} = 6656$ Tokens, 50 Trials per Pool Size M)

Can- didates (M)	**Dione Mean (μs)**	**Dione p95 (μs)**	**Exact DP (μs)**	**Speedup vs. DP**	**Optimality $R(S^*)/R(\mathbf{OPT})$**	**Gain vs. FIFO**
10	18.91	37.55	572.42	30.3×	99.48%	+23.1%
25	20.30	32.69	896.12	44.1×	99.26%	+89.6%
50	40.59	62.56	1,864.26	45.9×	99.71%	+148.4%
100	74.02	119.22	3,628.95	49.0×	99.96%	+228.3%
250	177.99	250.62	8,103.09	45.5×	100.00%	+426.5%
500	346.29	485.25	16,929.91	48.9×	100.00%	+556.7%

4.2 Experiment 2: 5-Tier Poset Permission Lattice & Indirect Injection Defense

We evaluated `SafetyKernel.validate_tool_call` across 500 tool call invocations spanning 8 operational and adversarial categories (20 iterations over 25 canonical test vectors covering benign reads, vision inspections, autonomous desktop actions, user-commanded multi-channel sends, unprompted indirect-injection sends, file deletions/process kills, adversarial prompt injections, and catastrophic OS commands).

Table 3: 5-Tier Poset Permission Lattice Gate Accuracy & Latency Across 500 Tool Evaluations

Safety Lattice Category	**Evaluations**	**Gate Accuracy**	**Mean (μs)**	**p95 (μs)**	**Kernel Enforcement Decision**
Tier 1: Read & Cloud Search	100	100.0%	39.04	52.81	Auto-Approved (`allowed=True`)
Tier 2: Screen & File Inspect	60	100.0%	40.70	84.40	Auto-Approved (`allowed=True`)
Tier 3: Desktop & Calendar Action	40	100.0%	44.10	73.08	Auto-Approved (`allowed=True`)
Tier 3: Intent-Verified Send	80	100.0%	42.56	65.08	Auto-Approved (User Intent Matched)
Tier 4: Unprompted Send Gate	60	100.0%	47.62	67.22	Paused for 1-Click Confirmation
Tier 4: Destructive OS/File Gate	60	100.0%	32.44	45.49	Paused for 1-Click Confirmation
Tier 4: Prompt Injection Gate	20	100.0%	36.85	42.64	Paused for 1-Click Confirmation
Tier 5: Catastrophic & Path Guard	80	100.0%	12.96	19.79	Permanently Hard-Blocked
Overall Safety Kernel	**500**	**100.0%**	**36.18**	**57.91**	**Zero False Negatives / Zero Leaks**

Table 3 confirms that the Poset Lattice and intent verifier achieve **100.0% classification accuracy** with a mean latency of **36.18 μs** (57.91 μs p95). Crucially, Tier 5 catastrophic commands finish fastest (12.96 μs) because Gate 1 short-circuits immediately upon matching compiled catastrophic regular expressions, and 100% of unprompted external communication calls (where the user prompt lacked communication intent) were intercepted at Tier 4.

4.3 Experiment 3: Hybrid MMR Memory Diversity, Knowledge Multigraph & Loop Breaker

We benchmarked `maximal_marginal_relevance` ($\lambda = 0.65$, $T_{1/2} = 30$ days, embedding dimension $d = 384$ matching `all-MiniLM-L6-v2`, $k = 5$) against standard top-k cosine similarity across clustered conversational candidate pools $N \in \{20, 50, 100, 250\}$ over 30 trials per size. We define *intra-list semantic diversity* among the k selected memory vectors $\{\mathbf{e}_{s_1}, \ldots, \mathbf{e}_{s_k}\}$ as the mean pairwise cosine distance:

$$\mathscr{D}(\mathscr{S}_k) = \frac{2}{k(k-1)} \sum_{1 \le a < b \le k} \big(1 - \cos(\mathbf{e}_{s_a}, \mathbf{e}_{s_b})\big). \tag{7}$$

As shown in Table 4, standard top-k cosine similarity suffers severe cluster collapse ($\mathscr{D} \approx 0.21$), returning 5 nearly identical conversational turns from the dominant topic cluster. Dione's decayed MMR retriever expands intra-list diversity to $\mathscr{D} \ge 0.92$ (**+320.5% to +344.5% gain**) in under 10 ms for $N \le 100$.

Table 4: Hybrid Temporal-Decay MMR Retrieval vs. Greedy Top-k Cosine Similarity ($d = 384, k = 5, \lambda = 0.65$)

Candidate Pool Size (N)	**Top-k**	**MMR Mean Latency (ms)**	**Greedy Cosine Diversity $\mathscr{D}$**	**Dione MMR Diversity $\mathscr{D}$**	**Diversity Gain (%)**
$N = 20$ candidates	5	1.737	0.2080	0.9246	+344.5%
$N = 50$ candidates	5	4.688	0.2081	0.9203	+342.2%
$N = 100$ candidates	5	9.580	0.2090	0.9251	+342.6%
$N = 250$ candidates	5	24.920	0.2191	0.9215	+320.5%

Finally, Table 5 summarizes the empirical performance of the Personal Knowledge Multigraph (`data/knowledge/graph.json`, comprising 60 entities, 11 typed edges, and 7 social dossiers) and the `ToolLoopDetector` circuit breaker.

Table 5: Knowledge Multigraph Traversal, Social Resolution & Loop Circuit Breaker Telemetry

Runtime Operation / Subsystem Metric	**Mean Latency**	**p95 Latency**	**Empirical Behavior / Verification**
2-Hop GraphRAG Query (`kg.query_relevant`)	665.00 μs	969.00 μs	Traverses 60 entities, 11 edges, 2-hop ego-graph
Entity Disambiguation (`disambiguate_entity`)	23.11 μs	31.40 μs	Distinguishes homonymous nodes (e.g., multiple "Alex")
Social Alias Resolution (`social.resolve_person`)	4.56 μs	6.80 μs	Resolves nicknames/roles to canonical email & phone
Loop Detector Hash Check (`loop.record_and_check`)	14.42 μs	26.70 μs	Trips at exact thresholds: repeat = 3, ping-pong = 8, max = 25

5 Conclusion & Reproducibility

We presented **Dione**, a sovereign Large Action Model orchestration engine that unifies local Windows 11 desktop automation, a 2-hop personal knowledge multigraph, and 8 external service integrations under rigorous mathematical guardrails. By pairing an $\mathcal{O}(M \log M)$ $\frac{1}{2}$-approximation Knapsack Context Allocator ($> 99.2\%$ empirical optimality in $< 350\,\mu$s) with an Intent-Aware 5-Tier Poset Permission Lattice (100% gate accuracy in 36.18 μs), temporal-decay MMR retrieval (+342% diversity), and closed-loop visual reflection, Dione achieves responsive, zero-mock autonomy without sacrificing user sovereignty or workstation safety. All benchmarks in this paper can be reproduced or extended with live Windows 11 GDI/UIA and Gemini API probes by executing `python server/bin/dione_research_benchmark.py -live-desktop -live-llm`.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *International Conference on Learning Representations (ICLR)*, 2023. Available: https://arxiv.org/abs/2210.03629

[2] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei, Y. Liu, Y. Xu, S. Zhou, S. Savarese, C. Xiong, V. Zhong, and T. Yu, "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024. Available: https://arxiv.org/abs/2404.07972

[3] S. Agashe, J. Han, S. Gan, J. Yang, A. Li, and X. E. Wang, "Agent S: An Open Agentic Framework that Uses Computers Like a Human," *arXiv preprint arXiv:2410.08164*, 2024. Available: https://arxiv.org/abs/2410.08164

[4] J. Yang, H. Zhang, F. Li, X. Zou, C. Li, and J. Gao, "Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V," *arXiv preprint arXiv:2310.11441*, 2023. Available: https://arxiv.org/abs/2310.11441

[5] J. Carbonell and J. Goldstein, "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries," in *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 335–336, 1998.

[6] H. Kellerer, U. Pferschy, and D. Pisinger, *Knapsack Problems*. Berlin, Germany: Springer-Verlag, 2004.

[7] D. E. Denning, "A Lattice Model of Secure Information Flow," *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, May 1976.

[8] Google AI for Developers, "Gemini Developer API Models, Pricing, and Rate Limits," *Google AI Studio Documentation*, Oct. 2026. Available: https://aistudio.google.com/docs/models and https://ai.google.dev/gemini-api/docs/rate-limits